

# Product Requirements Document: Skill Telemetry and Upkeep System for Agent-based Coding

## 1. Executive Summary

Large language model (LLM) agents used for coding rely on *skills*—bundled instructions stored in the repository—to handle tasks such as documentation, refactoring and testing. These skills are invoked automatically based on the current context and the contents of `AGENTS.md` files; more local `AGENTS.md` entries override instructions from parents in the directory tree <sup>1</sup>, so guidance is cumulative but nearest files take priority <sup>2</sup>. While this architecture lets teams package domain expertise, there is little visibility into how often each skill runs, whether it succeeds, or when it silently fails. Without instrumentation, skills rot as dependencies change, prompting instructions drift, and prompt-injection attacks may persist across sessions <sup>3</sup> or even embed spyware into long-term memory <sup>4</sup>. Recent studies on agentic pull requests show that agents often produce large, multi-purpose PRs that require human review and modifications; they are accepted about 84 % of the time but still trigger skepticism and rejections when design is suboptimal or trust is lacking <sup>5</sup> <sup>6</sup>.

This PRD proposes a **Skill Telemetry and Upkeep System** that instruments every skill invocation, logs granular events, periodically summarizes usage and failures, and generates reviewable proposals for improvement. A **logger skill** records structured events for each call; a **summarizer skill** analyzes logs to detect dead, flaky or high-value skills; and a **patch-applier skill** turns improvement proposals into pull requests. All updates are gated through version control, subject to human review and regression tests, and conform to robust security practices (input sanitization, cryptographic log integrity, segmentation and heuristic monitoring) drawn from the OWASP LLM prompt-injection guidance <sup>3</sup> and the SHIELD threat-mitigation framework <sup>7</sup>. The system separates logging, summarization and mutation, ensuring that skills remain stable while the agent evolves.

## 2. Problem Statement

### 2.1 Pain points

1. **Untracked skill usage.** In current agent systems, skills are invoked implicitly; there is no systematic telemetry to record when a skill runs, what parameters it received, which tools were called or whether the invocation succeeded. Engineers cannot tell which skills drive most value or which are rarely used.
2. **Silent skill rot and drift.** Skills embed assumptions about APIs, file structure, and style that may change. Without logs, a failing skill may quietly degrade or become brittle, causing the agent to regress or produce hallucinations. Agent-generated pull requests are often multi-purpose, touching documentation, tests and code simultaneously <sup>8</sup>; reviewers struggle to infer which skill caused an issue.

3. **Lack of prioritization.** With no usage metrics, maintainers cannot prioritize which skills need improvement. Dead skills waste tokens; flaky skills erode trust; high-value skills deserve investment.
4. **Prompt-injection persistence and safety.** Attackers can inject instructions into context windows or long-term memory, causing persistent malicious behaviors <sup>3</sup> <sup>4</sup> . Without strict separation between logs and instructions, telemetry may inadvertently feed hostile content back into skills.
5. **Uncontrolled self-modification.** Self-improving agents demonstrate the ability to edit their own code and gain 17–53 % performance improvements <sup>9</sup> , but safety considerations require human oversight and observability <sup>10</sup> . Updating skills directly from reflection loops risks runaway changes and prompt-injection persistence.

## 2.2 Non-goals

- The system does **not** design a new LLM or modify the `AGENTS.md` scoping semantics; it respects existing file-tree precedence <sup>1</sup> .
- It does **not** eliminate human review; all skill changes must be reviewed via pull request and pass existing tests <sup>11</sup> .
- It does **not** provide long-term model retraining; improvements target skill prompts and metadata rather than model weights.
- It does **not** include UI dashboards; rather it produces structured outputs that external dashboards can ingest.

## 3. Design Principles

1. **Determinism over magic.** Skills should be explicit, versioned and discoverable. The system must not rely on hidden memory or untracked state, avoiding the persistent memory attack vector <sup>4</sup> .
2. **Propose rather than apply.** Every suggestion for improving a skill must be represented as a proposal document or pull request; nothing edits code without review. GitHub best practices emphasise mandatory human code review, stricter CI checks and additional approvals for AI-generated code <sup>12</sup> .
3. **Observability-first.** Logging, metrics and traces should be treated as first-class citizens. Truefoundry's observability guidelines identify four pillars—step-level traces, tool latency, error propagation and loops <sup>13</sup> —and emphasise that metrics without context mislead <sup>14</sup> . The system instruments each skill invocation and tool call accordingly.
4. **Separation of concerns.** Logging, summarization and mutation are separate components. Logs are append-only and immutable; summarizers consume logs to produce summaries; patch-applier skills produce proposals. This aligns with the SHIELD framework, which recommends segmentation and logging immutability to mitigate agentic threats <sup>7</sup> .
5. **Fail closed.** On unhandled errors, the agent should surface failures and generate proposals rather than silently retry indefinitely. Self-reflection patterns can improve quality <sup>15</sup> , but they break down without explicit logging and evaluation <sup>16</sup> ; thus the system prefers explicit error reporting.

6. **Privacy and safety.** Logs may contain prompts and outputs, which might include personal or sensitive data. Telemetry must anonymize user identifiers, filter dangerous strings, and avoid feeding untrusted content back into prompts, in line with OWASP input-sanitization guidance <sup>17</sup>. The summarizer must treat logs as data, not instructions.

## 4. System Architecture

### 4.1 Components

1. **Repository tree with AGENTS.md.** The root AGENTS.md defines project-wide policies (style guides, architectural constraints, forbidden directories). According to the AGENTS.md specification, agents automatically read the nearest AGENTS.md file; instructions from lower directories override those from ancestors <sup>1</sup> <sup>2</sup>. Child files should include only directory-specific instructions while inheriting the root guidance <sup>18</sup>.
2. **Skills.** Each skill lives in its own folder under skills/skill\_name/ and contains:
  3. SKILL.md with front-matter (name, description, version) and markdown instructions.
  4. Optional AGENTS.md overriding global rules for that skill.
  5. Test files under tests/ verifying behaviour. Skills differ from tools: they modify the agent's context rather than executing code; they are invoked implicitly when their descriptions match the situation <sup>19</sup>, while tools perform side-effects with defined inputs and outputs <sup>20</sup>.
6. **Logger skill (instrumentation).** A global logger skill wraps every skill invocation and tool call. It records a structured event containing:
  7. **Identifiers:** unique invocation ID, parent invocation ID, session ID.
  8. **Metadata:** skill name, version, AGENTS.md hierarchy, timestamp.
  9. **Inputs and outputs:** sanitized prompts and results (truncated to avoid PII), tool call parameters and outputs.
  10. **Outcome:** status ( success, failure, error\_type ), retry count, exceptions.
  11. **Performance:** latency, token counts, model identifiers, tool latency.
  12. **User signals:** user corrections (when the user rephrases or edits agent output), overrides, or manual interventions.

Events are appended to an append-only file ( logs/YYYY-MM-DD.jsonl ) or streamed via an event bus. Each record is in JSON with optional OpenTelemetry trace fields (span ID, trace ID). Logs must be cryptographically signed or hashed to ensure integrity <sup>21</sup>.

1. **Telemetry store.** A storage subsystem persists logs. Options include:
  2. **JSONL files:** simple and diffable; easy to commit or upload. Suitable for low-volume environments but may grow large.
  3. **Event stream (Kafka/Redpanda):** scalable; supports subscribers for real-time dashboards. More operational complexity.
  4. **OpenTelemetry tracing:** integrated with existing observability tooling. Allows cross-service tracing; recommended for production systems <sup>22</sup>.

Logs must reside outside of the prompt context; they are not loaded by the agent and cannot contain instructions. Only the summarizer reads logs.

1. **Summarizer skill (upkeep).** A periodic summarizer job reads accumulated logs and produces:
2. `summaries/YYYY-MM-DD.md`: a human-readable overview of skill usage statistics (counts, success/failure ratios, median latency, token costs) and highlights of notable errors, loops and user corrections. It also cross-references skill versions and AGENTS.md overrides.
3. `backlog.json`: a machine-readable list of improvement proposals ranked by severity (dead skills, flaky skills, high-value skills). Each proposal includes rationale (e.g., "Skill X has 70 % failure rate over last week; error: missing package"), suggested fix (update prompt, update version, add test), and required approval level.
4. **Optional metrics file:** CSV or metrics that external dashboards can ingest.
5. **Patch-applier skill (proposal generator).** Triggered by backlog entries, this skill takes a proposal and constructs a candidate change to `SKILL.md` or tests. It does not commit directly; instead it opens a pull request using the repository's normal workflow. The PR includes:
  6. The proposed change (diff) and updated version number.
  7. A summary of telemetry supporting the change (from `summaries` and logs).
  8. Checklists for maintainers (CI status, tests passed, security scans).

The patch-applier must respect gating rules (Section 7).

1. **Maintainer / human reviewer.** The human reviewer evaluates PRs, ensures business logic correctness, verifies edge cases and error handling, and decides whether to merge. Community guidelines stress that if reviewers cannot explain AI-generated code, it should not ship <sup>23</sup>. Maintainers can modify proposals, request changes, or reject them.
2. **External dashboards (optional).** Teams may build Grafana, Datadog or custom dashboards that subscribe to the event stream or ingest the metrics file. Observability platforms like Langfuse or Arize provide UI for traces and evaluation metrics <sup>24</sup> <sup>25</sup>.

## 4.2 Data flow

```
[Skill invocation] → Logger skill records event → Event stored in telemetry →  
Summarizer reads logs periodically → Summary & backlog produced → Patch-applier  
generates proposal → Pull request opened → Human review & merge → Updated skill  
version deployed
```

## 4.3 Interaction with `AGENTS.md`

- The logger does not change instruction discovery. When a skill runs, the agent builds its context by concatenating the root `AGENTS.md`, any intermediate `AGENTS.md` and the skill's `SKILL.md`. Local files override ancestors <sup>1</sup>; therefore the summarizer must record which instruction sources were active for each invocation.

- The summarizer should detect cases where skills rely on instructions outside their directory (e.g., due to overlapping `AGENTS.md`) and flag potential scoping issues that cannot be enforced by directory-scoped files alone. For instance, rules like “no new dependencies” must be enforced by tests and CI, not by `AGENTS.md` alone <sup>26</sup>.

## 5. Telemetry Specification

### 5.1 Event Schema

Each log event is a JSON object with the following fields:

| Field                        | Type                                                                      | Required? | Description                                                                                                                         |
|------------------------------|---------------------------------------------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>event_id</code>        | string                                                                    | required  | UUID for this invocation.                                                                                                           |
| <code>parent_id</code>       | string                                                                    | optional  | Links nested skill or tool invocations.                                                                                             |
| <code>timestamp</code>       | ISO-8601 string                                                           | required  | Time of invocation.                                                                                                                 |
| <code>session_id</code>      | string                                                                    | optional  | Groups events belonging to the same user session.                                                                                   |
| <code>skill_name</code>      | string                                                                    | required  | Name of the skill invoked.                                                                                                          |
| <code>skill_version</code>   | string                                                                    | required  | Version declared in <code>SKILL.md</code> .                                                                                         |
| <code>agents_md_stack</code> | array of strings                                                          | required  | Ordered list of <code>AGENTS.md</code> files that contributed instructions (root first).                                            |
| <code>inputs</code>          | object                                                                    | optional  | Sanitized parameters passed to the skill.                                                                                           |
| <code>outputs</code>         | object                                                                    | optional  | Sanitized outputs returned by the skill.                                                                                            |
| <code>status</code>          | enum ( <code>success</code> , <code>failure</code> , <code>error</code> ) | required  | Outcome.                                                                                                                            |
| <code>error_type</code>      | string                                                                    | optional  | If status = <code>error</code> , classification (e.g., <code>timeout</code> , <code>exception</code> , <code>tool_failure</code> ). |
| <code>latency_ms</code>      | integer                                                                   | required  | Total time from invocation to completion.                                                                                           |
| <code>model_tokens</code>    | integer                                                                   | optional  | Number of tokens consumed.                                                                                                          |
| <code>tool_calls</code>      | array of objects                                                          | optional  | For each tool invocation: name, inputs, outputs, latency, status.                                                                   |
| <code>user_correction</code> | boolean                                                                   | optional  | True if user corrected or overrode the agent after this invocation.                                                                 |

| Field | Type   | Required? | Description                                 |
|-------|--------|-----------|---------------------------------------------|
| notes | string | optional  | Additional context; truncated to avoid PII. |

## 5.2 Success and failure definitions

- **Success:** the skill completes without exceptions and returns the expected type of result. Tools must have `status=success`. The summarizer may include additional heuristics such as user satisfaction (no subsequent corrections within the same session).
- **Failure:** the skill returns an explicit error or raises an exception; any tool call fails; or the agent times out or exceeds token limits. Repeated retries are logged as separate events with `parent_id` referencing the original invocation.

## 5.3 Storage and retention

- **JSONL:** default storage for small teams; commit logs under `logs/` with daily rotation. Commit history provides immutability and versioning.
- **Event stream:** recommended for high-volume systems; events written to Kafka/Redpanda with topic retention configurable (e.g., 30 days). Consumers include summarizer and dashboards.
- **Tracing:** optionally export events as OpenTelemetry spans; integrate with existing monitoring platforms <sup>22</sup>.
- **Retention & privacy:** logs should be retained long enough to support debugging and improvement cycles (e.g., 90 days) and then archived or deleted. Logs must anonymize user identifiers and redact sensitive inputs. When summarizer compiles backlogs, it must treat logs as data, not instructions, to avoid injection persistence <sup>3</sup>.

# 6. Upkeep / Summarizer Specification

## 6.1 Scheduling

- **Periodic run:** by default daily at off-peak hours via cron or event schedule. Frequency can be configured. Additional triggers: when logs exceed size threshold or after a major release.
- **Environment isolation:** run summarizer in a separate process with no tool execution privileges; it reads logs and writes summaries/proposals. This segmentation aligns with SHIELD's workload isolation strategy <sup>7</sup>.

## 6.2 Inputs

- All log events from the previous period.
- Current repository state (skills, versions, tests) to link logs to code.
- Historical summaries and backlog to compare trends.

## 6.3 Outputs

1. **Summary report** (`summaries/YYYY-MM-DD.md`)

The report should include:

- **Global metrics:** total skill invocations, success rate, average latency, total tokens consumed.
- **Per-skill metrics:** invocations, success/failure counts, median latency, tool usage, average tokens, error distribution.
- **Anomalies:** unusual error spikes, slowdowns, repeated retries. Use heuristics to detect infinite loops or context abandonment, as recommended by observability best practices <sup>13</sup>.
- **Top skills:** ranked by invocation count and by time saved (success × latency). Indicate high-value skills for further investment.
- **Flaky skills:** skills with high failure rates or high variance in outcomes. Provide error examples and associated inputs (redacted).
- **Dead skills:** skills not invoked during the period. Suggest deprecation.
- **User corrections:** list skills that trigger frequent user overrides (proxy for unsatisfactory performance).
- **Backlog** ( `backlog.json` )

JSON array of proposals; each entry has:

- `skill_name` and `version`.
- `issue_type`: `dead`, `flaky`, `improvement`.
- `evidence`: references to log events and summary metrics.
- `suggestion`: natural language description of what should change (e.g., “update API endpoint from v1 to v2; adjust prompt to remove outdated instructions”). If improvement arises from self-reflection or evaluation loops, include evaluation score.
- `priority`: numeric score based on failure rate, usage frequency, and token cost.
- `requires_review`: `true` for changes that could affect functionality, `false` for trivial metadata updates.
- **Metrics file (optional)**

A CSV or JSON document with per-skill metrics for ingestion into dashboards.

## 6.4 Heuristics

- **Dead skills:** no invocations in the last  $N$  days (configurable). Candidate for deprecation.
- **Flaky skills:** failure rate > threshold (e.g., 30 %) or high variance; flagged in backlog.
- **High-value skills:** high invocation count, high success rate, large token usage; prioritise for optimization (e.g., compress prompts, restructure to reduce token cost).
- **Error signatures:** cluster error messages to detect common failure modes; treat repeated tool exceptions or API errors as high priority.
- **User correction indicator:** if the agent’s output is immediately corrected by the user (observed through subsequent conversation or manual edits), mark the skill as requiring improvement.
- **Latency outliers:** detect skills that exceed latency thresholds; may indicate heavy tool calls.

## 7. Skill Update Workflow

1. **Proposal generation.** The summarizer adds entries to `backlog.json`. A patch-applier skill reads each entry and decides whether a proposal is safe to auto-generate. **Safe** changes include fixing typos, updating API URLs, adding clarifying comments, or adjusting version numbers. **Unsafe** changes (e.g., altering business logic, changing data persistence, modifying security-critical code) require explicit human authoring.
2. **Proposal document.** For each backlog entry, the patch-applier creates a proposal file (`proposals/skill_name/DATE.md`) summarizing the problem, evidence, and suggested changes. It attaches relevant log excerpts (truncated and redacted) and summary metrics.
3. **Pull request creation.** The patch-applier uses a CLI agent (similar to the CLI agents described in the Medium article, which integrate into CI pipelines and open PRs <sup>27</sup>) to open a PR with the proposed change. The PR includes the new `SKILL.md` version and updated tests (if needed). The commit message cites the summary and logs.
4. **Gating rules.**
5. **Human review mandatory.** The GitHub discussion emphasises that AI-generated code must undergo mandatory human review with no rubber-stamping; reviewers must verify business logic, edge cases, error handling and API contracts <sup>28</sup>. If the team cannot explain the AI's code, it does not ship <sup>29</sup>.
6. **Stricter CI.** AI PRs face stricter checks: unit/integration tests, static analysis, security scanning (CodeQL), dependency scanning, formatting gates <sup>30</sup>. Optionally require extra approvals or higher coverage thresholds <sup>31</sup>.
7. **Provenance tracking.** Label AI-generated PRs and store agent prompts, tool versions and model identifiers <sup>32</sup>. Tools like Git AI can provide line-level attribution.
8. **Segmentation of responsibilities.** For multi-agent systems, assign each agent one responsibility (scaffolding, tests, refactoring) and avoid agents editing the same files concurrently <sup>33</sup>. Humans resolve conflicts.
9. **Regression testing.** Before merging, run existing tests and new tests proposed by the summarizer. If any fail, the PR is blocked.
10. **Security gating.** Do not allow AI to modify security-critical modules (auth, payments, infrastructure) without senior review <sup>34</sup>.
11. **Version bump.** Each accepted change increments the skill's version number; the summarizer must record version usage.
12. **Post-merge actions.** After merging, the summarizer notes the updated version and resets failure counts. The logger continues to record events for the new version.



## 8. Repo Layout & Conventions

```

.
├── AGENTS.md                # root guidance, project-wide style/
constraints
├── skills/
│   ├── <skill_name>/
│   │   ├── SKILL.md        # skill instructions with front-matter
│   │   ├── AGENTS.md       # optional, skill-scoped overrides
│   │   ├── tests/          # automated tests verifying skill behaviour
│   │   └── ...              # supplementary files (e.g., templates,
scripts)
│   └── ...
├── logs/
│   └── 2026-01-17.jsonl     # daily JSONL logs (append-only)
├── summaries/
│   └── 2026-01-17.md        # daily human-readable summary
├── backlog.json             # machine-readable backlog of improvement
proposals
├── proposals/
│   └── <skill_name>/
│       └── 2026-01-18.md    # proposals for skill update
├── scripts/
│   ├── logger.py            # logger skill implementation
│   ├── summarizer.py        # summarizer skill implementation
│   └── patch_applier.py      # PR-creating skill
└── tests/                   # global tests verifying integrated behaviour

```

Naming conventions: skill names are lowercase with hyphens. Logs use ISO-date filenames. Summaries and proposals include dates. Telemetry lives under `logs/`; it is never loaded into context. Only the summarizer and patch-applier update files under `summaries/`, `backlog.json` and `proposals/` automatically. `AGENTS.md` files, `SKILL.md` files and code require PR-based changes.

## 9. Risks & Mitigations

| Risk                                | Description                                                             | Mitigation                                                                                                                                  |
|-------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Logging overhead and latency</b> | Instrumentation adds latency; capturing every tool call increases cost. | Batch events and use asynchronous logging; sample low-impact events. Use event streaming for high-volume systems.                           |
| <b>Privacy leak via logs</b>        | Logs may capture sensitive inputs or user identifiers.                  | Anonymize user IDs; limit stored input length; use sanitization filters to remove dangerous patterns <sup>17</sup> ; avoid storing secrets. |

| Risk                                        | Description                                                                                                                                                                         | Mitigation                                                                                                                                                                                                                                                             |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prompt injection in logs</b>             | Logs could contain malicious instructions that, if fed back into prompts, lead to prompt injection persistence <sup>3</sup> .                                                       | Treat logs as data, not instructions; summarizer and patch-applier must sanitize and never execute log content; structured prompts must clearly separate system instructions and user data <sup>35</sup> .                                                             |
| <b>Skill drift and runaway auto-editing</b> | Automated proposals may diverge from project goals or degrade quality. Self-improving agents show improved performance but require oversight <sup>10</sup> .                        | Only propose changes; require human review; enforce gating rules; maintain tests; restrict modifications to non-critical areas.                                                                                                                                        |
| <b>Conflict between agents</b>              | Multiple agents editing the same files leads to chaos <sup>33</sup> .                                                                                                               | Assign one responsibility per agent; schedule sequential runs; implement locking on skill directories.                                                                                                                                                                 |
| <b>Operational complexity</b>               | Event streams, tracing, cryptographic log signing and summarization increase operational burden.                                                                                    | Provide default JSONL fallback; allow teams to opt into advanced features; document infrastructure requirements.                                                                                                                                                       |
| <b>Attack surface</b>                       | GenAI agents introduce unique threats (cognitive architecture, temporal persistence, operational execution, trust boundary violations and governance circumvention) <sup>36</sup> . | Apply SHIELD strategies: segmentation, heuristic monitoring, integrity verification, escalation control and logging immutability <sup>7</sup> . Limit agents' capabilities via least-privilege; isolate skills and tools; run security scans on AI PRs <sup>37</sup> . |
| <b>Lack of calibration</b>                  | Heuristics for dead/flaky skills may misclassify or produce noise.                                                                                                                  | Tune thresholds over time; allow maintainers to adjust heuristics; include human judgement loops. Compare with evaluation frameworks like self-evolving agent loops <sup>38</sup> .                                                                                    |

## 10. Open Questions & Assumptions

1. **Threshold calibration.** What failure rate defines a flaky skill? The PRD assumes 30 % but this needs empirical tuning per project.
2. **Event retention.** How long should logs be retained? The PRD assumes 90 days; sensitive applications might require shorter retention.
3. **Scoping limitations.** Directory-scoped `AGENTS.md` files may not enforce all rules (e.g., "no new dependencies"). Additional enforcement via CI and tests is assumed <sup>26</sup>.

4. **Automatic summarizer frequency.** The default daily run may not suit all projects. Real-time summarization could be useful but increases resource use; periodic summarization may miss immediate failures.
5. **Integration with evaluation loops.** Self-evolving workflows propose using LLM-as-judge for automated prompt refinement <sup>39</sup>. This PRD assumes human-in-the-loop proposals. Future work could integrate evaluation frameworks with summarizer outputs.
6. **Telemetry cost and token usage.** Recording token counts and storing prompts increases cost and raises privacy concerns. The PRD assumes token counting instrumentation is available; this may vary across LLM providers.
7. **Tool instrumentation.** Tools may already produce logs; duplication should be avoided. The logger must interoperate with existing observability pipelines.
8. **User feedback signals.** The PRD infers user corrections by analysing conversation patterns; this is a heuristic and may require user-interface integration.
9. **Scope of security mitigations.** The SHIELD framework lists heuristic monitoring and logging immutability but requires sophisticated infrastructure <sup>7</sup>. The PRD assumes teams can implement at least basic segmentation and input sanitization; advanced heuristics may be optional.

---

## References

- AGENTS.md specification and scoping rules <sup>1</sup> <sup>2</sup>
- Difference between skills and tools <sup>19</sup> <sup>20</sup>
- Reflexion and self-reflection patterns <sup>15</sup> <sup>16</sup>
- Agent observability guidelines <sup>13</sup> <sup>14</sup> <sup>22</sup>
- Importance of AI logging components <sup>40</sup> <sup>41</sup>
- Background agents handling upkeep tasks <sup>5</sup>
- Empirical data on agentic pull requests <sup>8</sup> <sup>6</sup> <sup>42</sup>
- Prompt-injection threats and persistence <sup>3</sup> <sup>4</sup>
- SHIELD mitigation framework for GenAI agents <sup>7</sup>
- Best practices for AI-generated code and PR gating <sup>12</sup> <sup>37</sup> <sup>33</sup> <sup>32</sup>
- Self-evolving agent feedback loop <sup>38</sup> <sup>39</sup>

---

<sup>1</sup> AGENTS.md

<https://agents.md/>

<sup>2</sup> <sup>18</sup> Proposal: AGENTS.md v1.1: Making Implicit Semantics Explicit, Clarifying Scope, and Recommendation for Progressive Disclosure · Issue #135 · agentsmd/agents.md

<https://github.com/agentsmd/agents.md/issues/135>

<sup>3</sup> <sup>17</sup> <sup>35</sup> LLM Prompt Injection Prevention - OWASP Cheat Sheet Series

[https://cheatsheetseries.owasp.org/cheatsheets/LLM\\_Prompt\\_Injection\\_Prevention\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html)

- 4 Spyware Injection Into Your ChatGPT's Long-Term Memory (SpAIware) · Embrace The Red  
<https://embracethered.com/blog/posts/2024/chatgpt-macos-app-persistent-data-exfiltration/>
- 5 27 AI Coding Tools: What Works (and What Doesn't) for Me | by Aleksandar Mirilovic | Medium  
<https://medium.com/@amirilovic/ai-coding-tools-what-works-and-what-doesnt-for-me-56be3d3c1018>
- 6 On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub  
<https://arxiv.org/html/2509.14745v1>
- 7 21 36 Securing Agentic AI: A Comprehensive Threat Model and Mitigation Framework for Generative AI Agents  
<https://arxiv.org/html/2504.19956v1>
- 8 42 Agents vs. Humans on GitHub: Who Is Actually Writing Code Today, and How | by Dataism Lab | Medium  
<https://medium.com/@dataism/agents-vs-humans-on-github-who-is-actually-writing-code-today-and-how-ea0bab7a60c4>
- 9 10 A Self-Improving Coding Agent  
<https://arxiv.org/html/2504.15228v2>
- 11 12 23 26 28 29 30 31 32 33 34 37 Best practices for using GitHub AI coding agents in production workflows? · community · Discussion #182197 · GitHub  
<https://github.com/orgs/community/discussions/182197>
- 13 14 AI Agent Observability: Monitoring and Debugging Agent Workflows  
<https://www.truefoundry.com/blog/ai-agent-observability-tools>
- 15 Hello Agentic AI: The Reflection Pattern - Making AI Systems Self-Correcting | by Alessandro | Jan, 2026 | Medium  
<https://medium.com/@alessandro.a.pagliaro/hello-agentic-ai-the-reflection-pattern-making-ai-systems-self-correcting-6e413109f323>
- 16 Reflexion | Prompt Engineering Guide  
<https://www.promptingguide.ai/techniques/reflexion>
- 19 Agent Skills - Claude Code Docs  
<https://code.claude.com/docs/en/skills>
- 20 Skills vs Tools for AI Agents: Production Guide  
<https://blog.arcade.dev/what-are-agent-skills-and-tools>
- 22 24 Mastering AI agent observability: A comprehensive guide | by Dave Davies | Online Inference | Medium  
<https://medium.com/online-inference/mastering-ai-agent-observability-a-comprehensive-guide-b142ed3604b1>
- 25 Monitoring AI Research Assistants - Ensuring Accuracy and Reliability - Langfuse  
<https://langfuse.com/faq/all/ai-research-assistant-monitoring>
- 38 39 Self-Evolving Agents - A Cookbook for Autonomous Agent Retraining  
[https://cookbook.openai.com/examples/partners/self\\_evolving\\_agents/autonomous\\_agent\\_retraining](https://cookbook.openai.com/examples/partners/self_evolving_agents/autonomous_agent_retraining)
- 40 41 What is AI logging or LLM logging or AI Application Logging | newline  
<https://www.newline.co/@Dipen/what-is-ai-logging-or-llm-logging-or-ai-application-logging--d7377b73>